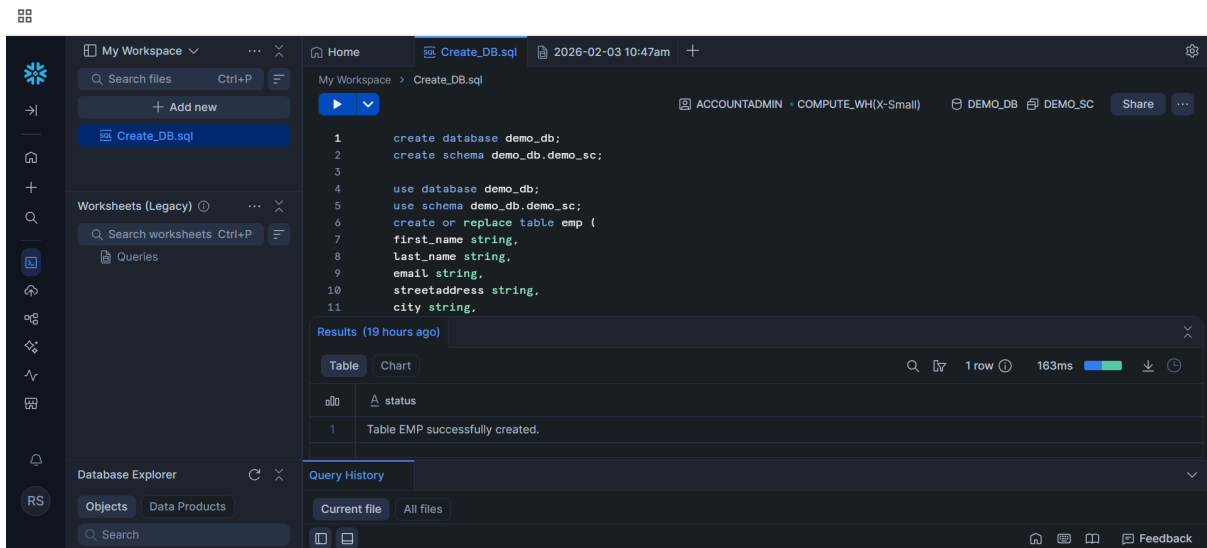


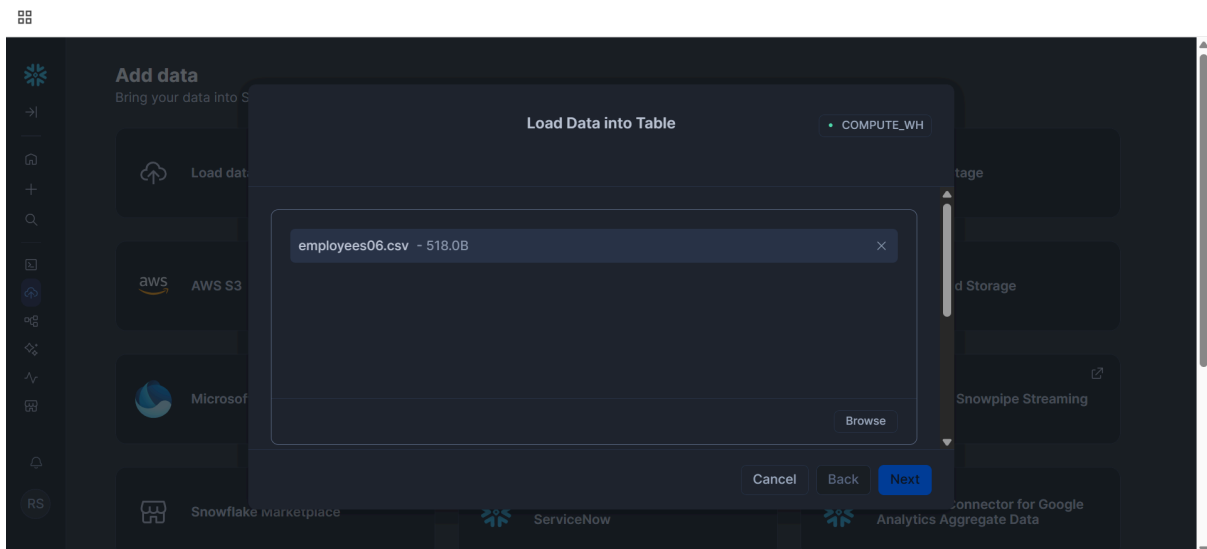
Snowflake Screenshots

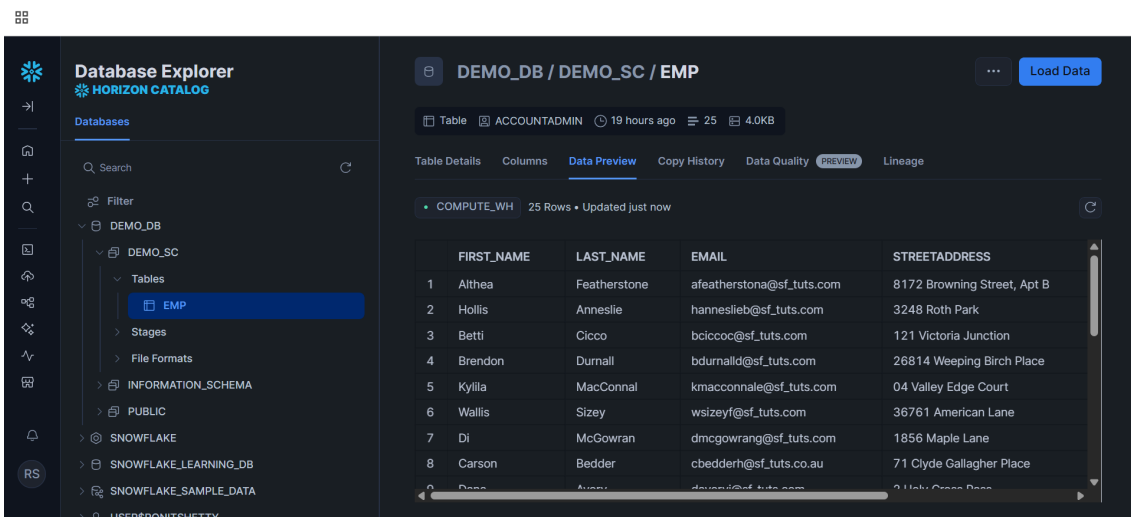
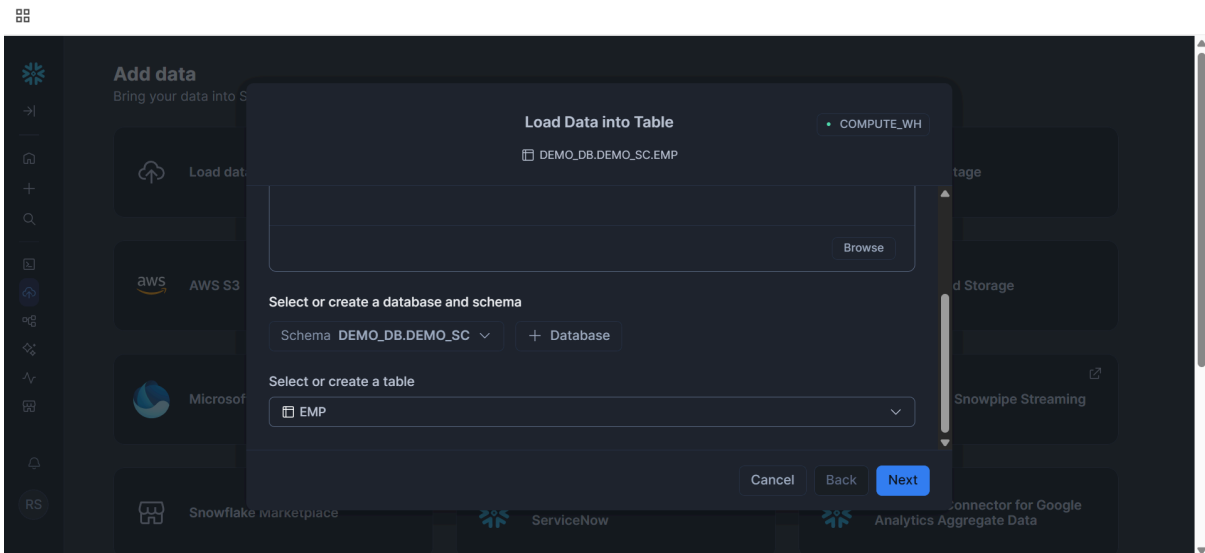
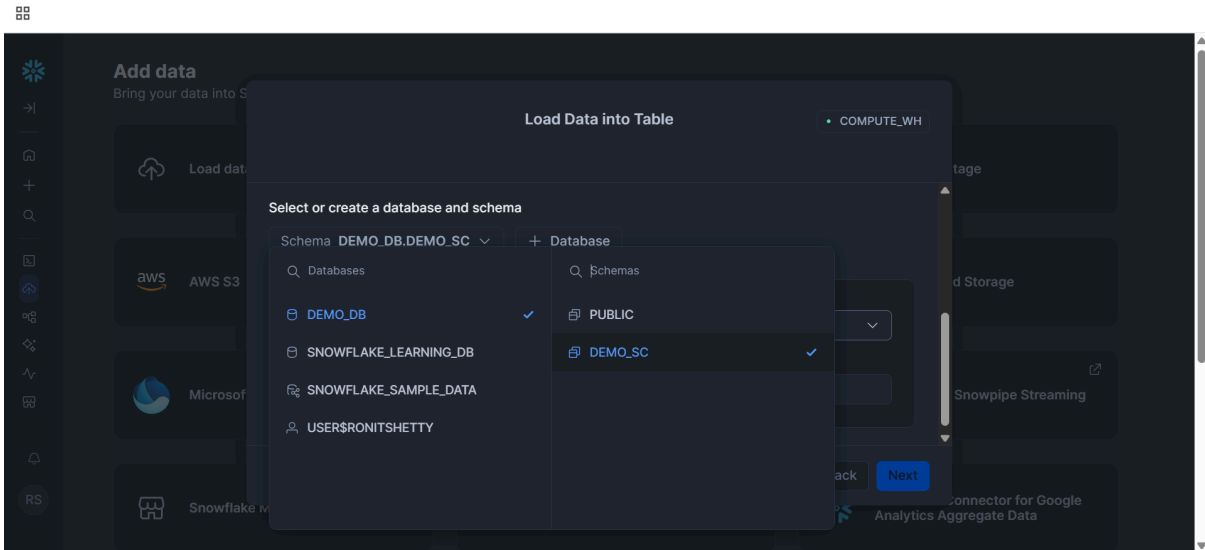
Name: Ronit Shetty

Creating database, schema and table



Loading data through web UI





Running queries



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Create_DB.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Home

Create_DB.sql

2026-02-03 10:47am

Queries

1 SELECT * FROM "DEMO_DB"."DEMO_SC"."EMP" LIMIT 10;

2

3 SELECT COUNT(*) AS total_employees FROM "DEMO_DB"."DEMO_SC"."EMP";

4

5 | Ctrl+I to generate

Results (1 minute ago)

Table Chart

1 row 46ms

Table

1 TOTAL_EMPLOYEES 25

Query History

Current file All files



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Home

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

1 SELECT

2 MONTHNAME(start_date) as hiring_month,

3 COUNT(*) as hires

4 FROM "DEMO_DB"."DEMO_SC"."EMP"

5 WHERE YEAR(start_date) = 2017

6 GROUP BY hiring_month

7 ORDER BY hires DESC;

8

9 SELECT

10 CONCAT(first_name, ' ', last_name) as full_name,

11 email

Results (3 minutes ago)

Table Chart

11 rows 115ms

Table

1 HIRING_MONTH 9.1% 9.1%

2 Apr 9.1%

3 Aug 9.1%

Query History

Current file All files



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Home

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

10 SELECT

11 CONCAT(first_name, ' ', last_name) as full_name,

12 email,

13 city

14 FROM "DEMO_DB"."DEMO_SC"."EMP"

15 ORDER BY last_name ASC;

Results (6 minutes ago)

Table Chart

25 rows 72ms

Table

1 FULL_NAME EMAIL CITY

2 Althea Featherstone 4.0% adavidovtsk@sf_tuts.com 4.0% Aleysk

3 Arlene Davidovits 4.0% afeatherstona@sf_tuts.com 4.0% Banjar Bengkelgede

4 +23 more

5 Hollis Anneslie hanneslieb@sf_tuts.com Aleysk

6 Dana Avory davoryi@sf_tuts.com Wenlin

7 Granger Bassford gbassfordo@sf_tuts.co.uk Karditsa

Query History

Current file All files



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

Basic_Queries.sql

My Workspace > Basic_Queries.sql

1 -- 1. Simple Filtering (WHERE)

2 -- Get all employees who live in 'Miami' or have 'Miami' in their address

3 SELECT * FROM "DEMO_DB"."DEMO_SC"."EMP"

4 WHERE city = 'Miami'

5 OR streetaddress LIKE '%Miami%';

6

7 -- 2. Sorting Data (ORDER BY)

8 -- List employees alphabetically by their Last Name

9 SELECT first_name, last_name, start_date

Results (just now)

Table Chart

1 row 2.0s

	FIRST_NAME	LAST_NAME	EMAIL	STREETADDRESS	CITY
1	Cesar	Hovie	chovie9@sf_tuts.com	5 7th Pass	Miami

Query History



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

Basic_Queries.sql

My Workspace > Basic_Queries.sql

4 WHERE city = 'Miami'

5 OR streetaddress LIKE '%Miami%';

6

7 -- 2. Sorting Data (ORDER BY)

8 -- List employees alphabetically by their Last Name

9 SELECT first_name, last_name, start_date

10 FROM "DEMO_DB"."DEMO_SC"."EMP"

11 ORDER BY last_name ASC;

12

13 -- 3. Aggregation (GROUP BY)

Results (just now)

Table Chart

25 rows 883ms

	FIRST_NAME	LAST_NAME	START_DATE
	Althea	Anneslie	
	Arlene	Avory	
	+23 more	+23 more	
1	Hollis	Anneslie	2017-11-16
2	Dana	Avory	2017-05-11

Query History



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

Basic_Queries.sql

My Workspace > Basic_Queries.sql

26

27 -- 5. Date Range Filtering

28 -- Find everyone hired between Jan 1, 2017 and June 1, 2017

29 SELECT first_name, last_name, start_date

30 FROM "DEMO_DB"."DEMO_SC"."EMP"

31 WHERE start_date BETWEEN '2017-01-01' AND '2017-06-01'

32 ORDER BY start_date;

Results (just now)

Table Chart

7 rows 82ms

	FIRST_NAME	LAST_NAME	START_DATE
1	Violette	Shermore	2017-01-19
2	Carson	Bedder	2017-03-29
3	Ivett	Casemore	2017-03-29
4	Nvssa	Doraan	2017-04-13

Query History



The screenshot displays the Snowflake web interface. On the left is a sidebar with navigation options: 'Work with data', 'Projects', 'Ingestion', 'Transformation', 'AI & ML', and 'Monitoring'. Below this is a credit balance section showing '\$398 credits left' and a trial end date of '29 days'. The main workspace is divided into three panes. The top pane shows a file explorer with 'My Workspace' containing 'Basic_Queries.sql', 'Create_DB.sql', and 'Sample_Queries.sql'. The middle pane shows a 'Worksheets (Legacy)' section with a search bar and a 'Queries' tab. The bottom pane shows the 'Database Explorer' with 'Objects' and 'Data Products' tabs. The right pane is the active area, showing a SQL query in the editor and its results in a table. The query is a SELECT statement that filters for employees in the 'IN (10, 11, 12)' group, orders by 'start_date', and uses a CASE WHEN statement to label employees as 'Veteran' or 'Newcomer' based on their 'days_employed'. The results table shows columns for 'FIRST_NAME', '#', 'DAYS_EMPLOYED', and 'STATUS'. The data is grouped by 'FIRST_NAME' and 'STATUS'. The 'Althea' group has 2976 days employed and is a 'Veteran'. The 'Arlene' group has 3334 days employed and is a 'Newcomer'. The 'Holis' group has 3127 days employed and is a 'Veteran'. The 'Newcomer' group has 3000 days employed and is a 'Newcomer'.

SQL Query:

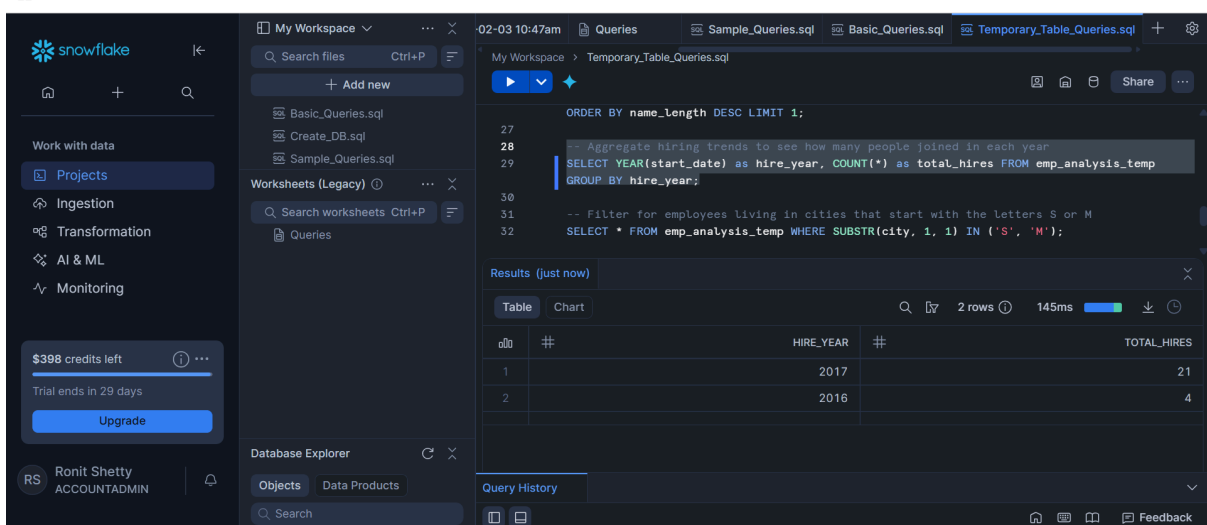
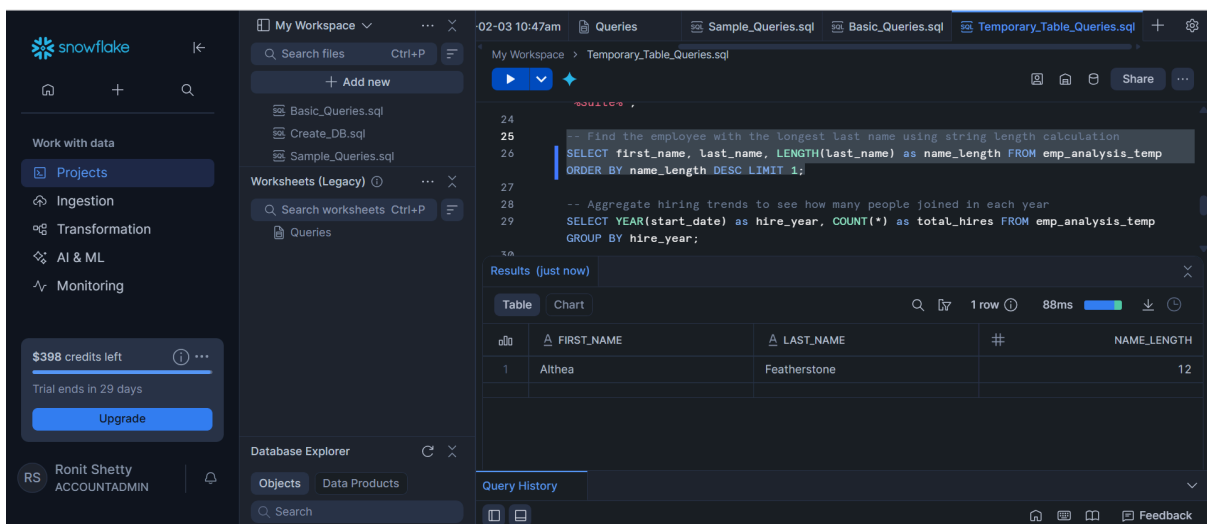
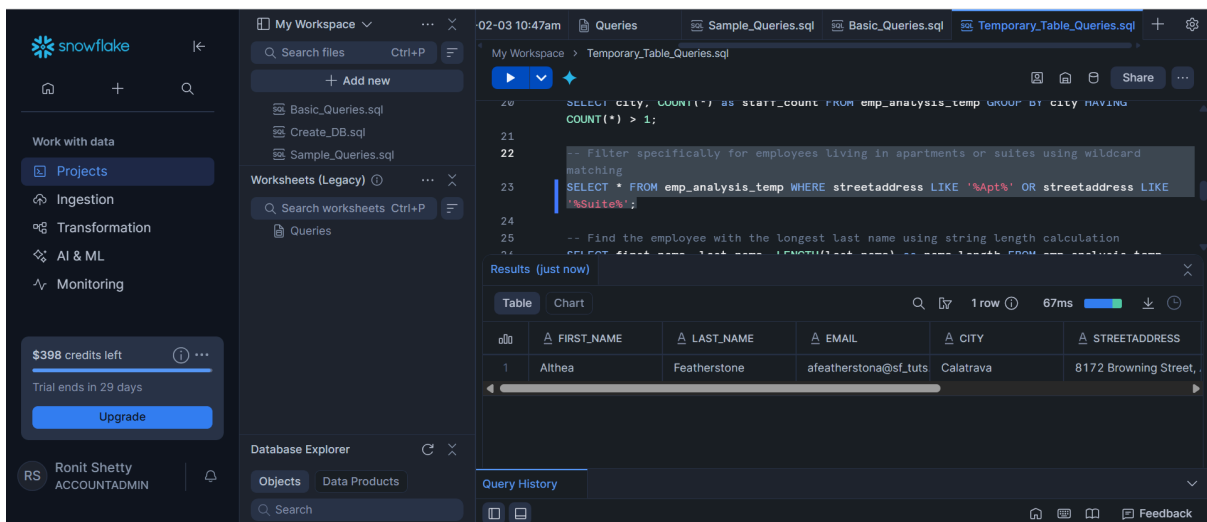
```
SELECT first_name, last_name, start_date FROM emp_employee WHERE emp_employee IN (10, 11, 12) ORDER BY start_date;

-- Use conditional logic to label employees as 'Veteran' or 'Newcomer' based on days employed
SELECT first_name, days_employed, CASE WHEN days_employed > 3000 THEN 'Veteran' ELSE 'Newcomer' END as status FROM emp_analysis_temp;

-- Calculate the average employment duration (in days) for each city
SELECT city, AVG(days_employed) as avg_tenure_days FROM emp_analysis_temp GROUP BY city
```

Results (just now)

Table	Chart	25 rows	67ms
Althea	4.0%	2976	3334
Arlene	4.0%		
+23 more			
1 Althea		3127	Veteran
2 Holis		3000	Newcomer



The screenshot shows the Snowflake SQL Editor interface. On the left is a sidebar with navigation options like 'Projects', 'Ingestion', 'Transformation', 'AI & ML', and 'Monitoring'. The main workspace contains a file explorer with 'Basic_Queries.sql', 'Create_DB.sql', and 'Sample_Queries.sql'. The 'Worksheets (Legacy)' section shows a search bar and a list of queries. The 'Database Explorer' at the bottom left shows 'Objects' and 'Data Products'. The main editor displays a SQL query in 'Temporary_Table_Queries.sql' with line numbers 28 to 32. The query is an aggregate query grouped by hire_year, filtered by city starting with 'S' or 'M'. Below the editor, the 'Results (just now)' section shows a table with 9 rows and 81ms execution time. The table has columns: FIRST_NAME, LAST_NAME, EMAIL, CITY, STREETADDRESS, and STAFF. The results show data for four employees: Betti Cicco, Brendon Durnall, Catherin Devereu, and Grazia Glaserman.

	FIRST_NAME	LAST_NAME	EMAIL	CITY	STREETADDRESS	STAFF
1	Betti	Cicco	bcicco@sf_tuts.com	Sinegor'ye	121 Victoria Junction	2017-0
2	Brendon	Durnall	bdurnald@sf_tuts.com	Sabadell	26814 Weeping Birch Pl	2017-1
3	Catherin	Devereu	cdevereu6@sf_tuts.co.au	Magapit	535 Basil Terrace	2016-1
4	Grazia	Glaserman	gglaserman7@sf_tuts.coi	Shiquanhe	162 Debra Lane	2017-0

Query caching enabled (default) - Result reuse

The screenshot shows the Snowflake Query Profile page for a specific query. The query ID is '01c229a9-0001-788a-0000-000d8baec619'. The user is 'RONITSHETTY'. The page has three tabs: 'Query Details', 'Query Profile' (selected), and 'Query Telemetry'. The 'Query Profile' tab shows a 'Query insights' section with 'None detected. Learn more'. Below that is a 'Profile overview(Finished)' section with a progress bar and a legend showing 'Other' at 100.0%. At the bottom, there is a 'QUERY RESULT REUSE [0]' section with a progress bar and a legend showing '01c229a9-0001-788a-0000-000d8bae...' at 100%.

QUERY RESULT REUSE [0]	100%
01c229a9-0001-788a-0000-000d8bae...	

Connecting to azure:

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and the user profile 'Ronit Shetty@ext.saint-...'. The main content area displays the 'container1 | Properties' page. On the left, there is a sidebar with navigation options: Overview, Diagnose and solve problems, Access Control (IAM), Settings, Shared access tokens, Access policy, Properties (selected), and Metadata. The main panel shows the following details for 'container1':

- NAME:** container1
- URL:** https://ronitshettystorage.blob.core.windows.net/container1
- LAST MODIFIED:** 2/3/2026, 1:16:27 PM
- ETAG:** 0x8DE62F856885B2
- LEASE STATUS:** Unlocked

A 'Copy' button is visible next to the URL field.

The screenshot shows the Snowflake SQL interface. The top navigation bar includes the Snowflake logo, a search bar, and the user profile 'Ronit Shetty ACCOUNTADMIN'. The main content area displays a query execution window. The query being executed is:

```
1 create or replace storage integration azure_int
2 type = external_stage
3 storage_provider = azure
4 enabled = true
5 azure_tenant_id = 'e339bd4b-2e3b-4035-a452-2112d502f2ff'
6 storage_allowed_locations = ('azure://ronitshettystorage.blob.core.windows.net/container1');
7
8 desc integration azure_int;
```

The results of the query are displayed in a table with 1 row and 1 column:

status
Integration AZURE_INT successfully created.

The interface also shows a 'Query History' section at the bottom.

Error with azure (snowsql is restricted too)

The screenshot shows a Saint-Gobain login page. The page features a background image of a factory floor with workers. A central white box contains the following text:

Need admin approval

otuesrsnowflakepacint
unverified

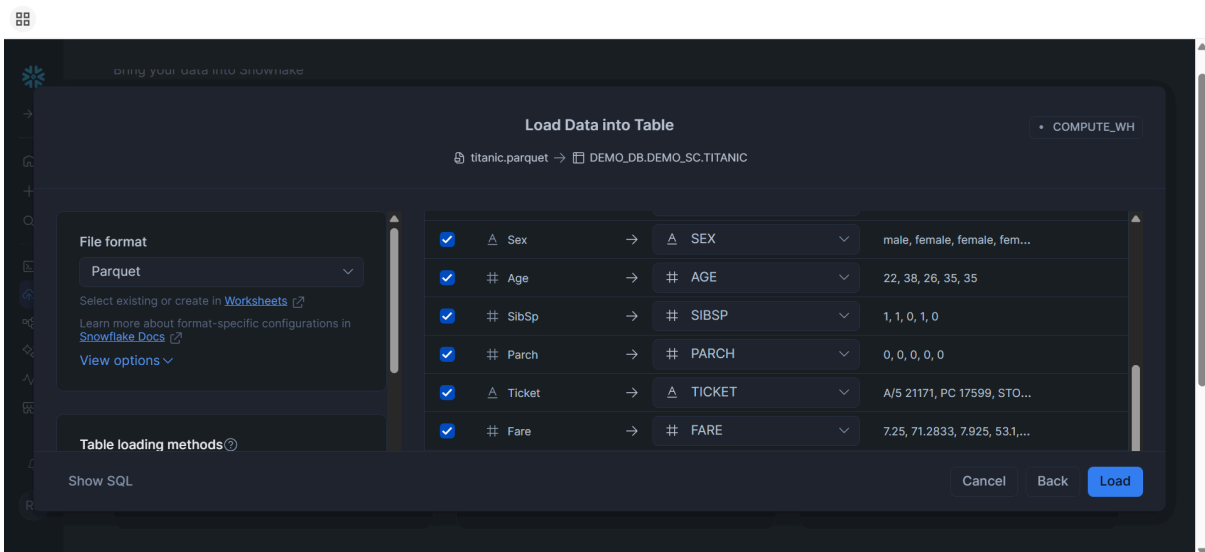
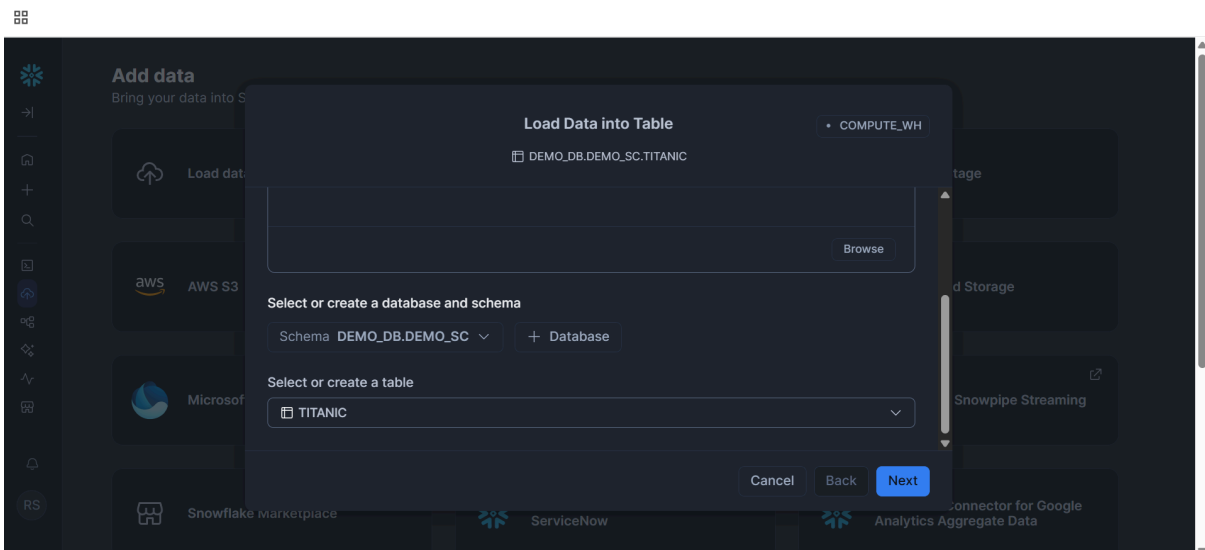
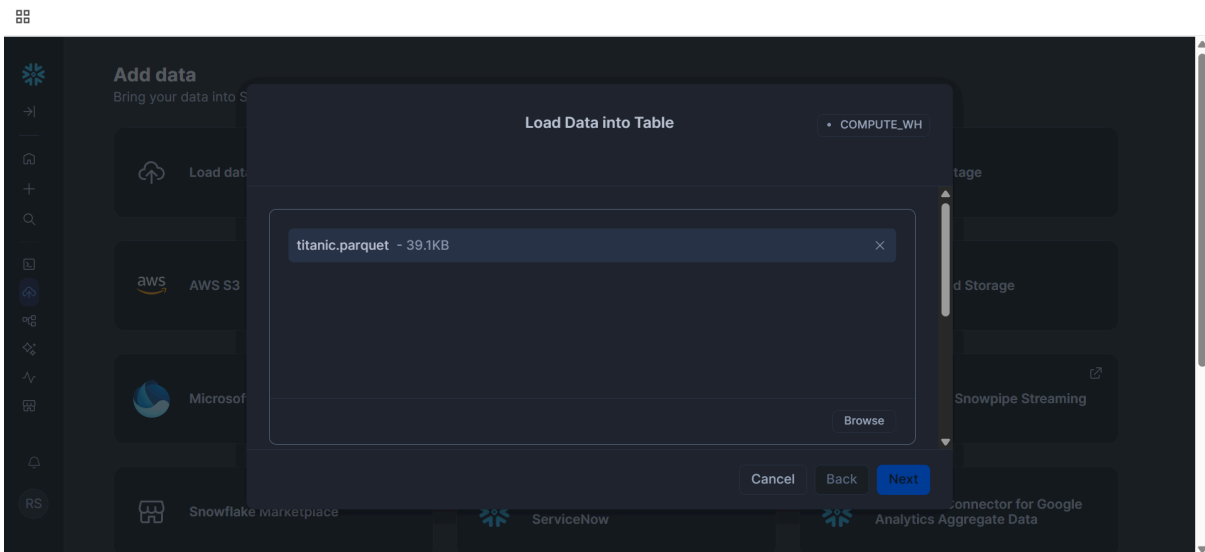
otuesrsnowflakepacint needs permission to access resources in your organization that only an admin can grant. Please ask an admin to grant permission to this app before you can use it.

[Have an admin account? Sign in with that account](#)

[Return to the application without granting consent](#)

The page also includes a 'Terms of use' link and a 'Privacy & cookies' link at the bottom right.

Titanic data loading





Database Explorer
HORIZON CATALOG

Databases

Search

Filter

DEMO_DB

DEMO_SC

Tables

EMP

TITANIC

Stages

File Formats

INFORMATION_SCHEMA

PUBLIC

SNOWFLAKE

SNOWFLAKE_LEARNING_DB

SNOWFLAKE_SAMPLE_DATA

DEMO_DB / DEMO_SC / TITANIC

Table ACCOUNTADMIN 6 minutes ago 0 0.0B

Table Details Columns Data Preview Copy History Data Quality (PREVIEW) Lineage

COMPUTE_WH 100 Rows • Updated just now

	PASSENGERID	SURVIVED	PCLASS	NAME
1	1	0	3	Braund, Mr. Owen Harris
2	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)
3	3	1	3	Heikkinen, Miss. Laina
4	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)
5	5	0	3	Allen, Mr. William Henry
6	6	0	3	Moran, Mr. James
7	7	0	1	McCarthy, Mr. Timothy J
8	8	0	3	Palsson, Master. Gosta Leonard



Database Explorer
HORIZON CATALOG

Databases

Search

Filter

DEMO_DB

DEMO_SC

Tables

EMP

TITANIC

Stages

File Formats

INFORMATION_SCHEMA

PUBLIC

SNOWFLAKE

SNOWFLAKE_LEARNING_DB

DEMO_DB / DEMO_SC / TITANIC

Table ACCOUNTADMIN 7 minutes ago 0 0.0B

Table Details Columns Data Preview Copy History Data Quality (PREVIEW) Lineage

Last 7 days Status All Pipe All 1 Copy Search COMPUTE_WH

Loaded (1)

1

0 Jan 28 Jan 29 Jan 30 Jan 31 Feb 1 Feb 2 Feb 3

FILE NAME	LOADED	SIZE	ROWS	STATUS	SOUR...
2026-02-03T11:10:46.952Z/titanic.par...	1m ago	39.1KB	891	Loaded	

Running queries



snowflake

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

load_to_azure.sql

Monthwise_Hires.sql

Temporary_Table_Queries.sql

Titanic_Data.sql

Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

DEMO_SC

Tables 2

Stages 1

My Workspace

Basic_Queries.sql

Temporary_Table_Queries.sql

Verifying_Data_Load.sql

load_to_azure.sql

Titanic_Data.sql

My Workspace > Titanic_Data.sql

24);

25

26 -- 3. Verify

27 SELECT * FROM titanic LIMIT 10;

Results (just now)

Table Chart

10 rows 66ms

#	PASSENGERID	#	SURVIVED	#	PCLASS	NAME	SEX	#	AGE	#	SURVIVED
1	1	0	1	3	Allen, Mr. William Henry	male	22	0			
2	2	1	1	1	Braund, Mr. Owen Harris	male	22	0			
3	3	1	1	3	Cumings, Mrs. John Bradley	female	38				
4	4	1	1	1	Heikkinen, Miss. Laina	female	26				

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
 - Tables 2
 - Stages 1

My Workspace > **Titanic_Queries.sql**

1 `SELECT COUNT(*) FROM demo_db.demo_sc.titanic;`

Results (just now)

Table	Chart	1 row	29ms
#		COUNT(*)	
1		891	

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
 - Tables 2
 - Stages 1

My Workspace > **Titanic_Queries.sql**

```
4 SELECT COUNT(*) FROM demo_db.demo_sc.titanic;
5
6 -- 2. Survival Count: How many survived (1) vs Died (0)?
7 SELECT Survived, COUNT(*) as passenger_count
8 FROM titanic
9 GROUP BY Survived;
10
11 -- 3. The "Women and Children First" Check
12 -- Calculate survival rate by Gender
13
```

Results (just now)

Table	Chart	2 rows	120ms
#		SURVIVED	# PASSENGER_COUNT
1		1	342
2		0	549

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
 - Tables 2
 - Stages 1

My Workspace > **Titanic_Queries.sql**

```
12 -- Calculate survival rate by Gender
13 SELECT
14     Sex,
15     COUNT(*) as total_people,
16     SUM(Survived) as survivors,
17     ROUND((SUM(Survived) / COUNT(*) * 100, 2) as survival_rate_percent
18 FROM titanic
19 GROUP BY Sex;
20
```

Results (just now)

Table	Chart	2 rows	80ms				
#	SEX	#	TOTAL_PEOPLE	#	SURVIVORS	#	SURVIVAL_RATE_PERCENT
1	female		314		233		74.20
2	male		577		109		18.89

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- DEMO_DB
 - DEMO_SC
 - Tables 2
 - Stages 1

My Workspace > Titanic_Queries.sql

```
22 SELECT
23     Pclass,
24     COUNT(*) as total_passengers,
25     SUM(Survived) as survivors,
26     ROUND((SUM(Survived) / COUNT(*)) * 100, 2) as survival_rate_percent
27 FROM titanic
28 GROUP BY Pclass
29 ORDER BY Pclass;
```

Results (2 minutes ago)

Table Chart 3 rows 95ms

#	PCLASS	#	TOTAL_PASSENGERS	#	SURVIVORS	#	SURVIVAL_RATE_PERCENT
1	1		216		136		62.96
2	2		184		87		47.28
3	3		491		119		24.24

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- DEMO_DB
 - DEMO_SC
 - Tables 2

My Workspace > Titanic_Queries.sql

```
32 SELECT
33     CASE WHEN Survived = 1 THEN 'Survived' ELSE 'Perished' END as status,
34     ROUND(AVG(Age), 1) as avg_age,
35     MIN(Age) as youngest,
36     MAX(Age) as oldest
37 FROM titanic
38 GROUP BY Survived;
```

Results (just now)

Table Chart 2 rows 716ms

#	STATUS	#	AVG_AGE	#	YOUNGEST	#	OLDEST
1	Perished		30.6		1		
2	Survived		28.3		0.42		

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- DEMO_DB
 - DEMO_SC
 - Tables 2
 - Stages 1

My Workspace > Titanic_Queries.sql

```
54 LIMIT 10;
55
56 -- 8. Embarkation: Where did most people get on?
57 -- C = Cherbourg, Q = Queenstown, S = Southampton
58 SELECT Embarked, COUNT(*) as count
59 FROM titanic
60 GROUP BY Embarked
61 ORDER BY count DESC;
```

Results (just now)

Table Chart 4 rows 90ms

#	EMBARKED	#	COUNT
1	S		644
2	C		168
3	Q		77
4	null		2

Query History

Creating new stage and using COPY for manual import

The screenshot shows the Snowflake SQL Editor interface. The top bar displays the workspace name 'My Workspace' and the active file 'Titanic_manual_load.sql'. The left sidebar contains a 'Database Explorer' panel with a search bar and a list of objects, including 'MY_INTERNAL_STAGE' and 'TITANIC_STAGE'. The main editor area shows a SQL query that creates a stage named 'titanic_stage' with a file format of 'PARQUET'. The query is as follows:

```
1 USE DATABASE demo_db;
2 USE SCHEMA demo_sc;
3
4 -- 1. Create a Specific Stage for this dataset
5 -- A dedicated stage helps keep your Titanic data isolated from other uploads.
6 CREATE OR REPLACE STAGE titanic_stage
7   FILE_FORMAT = (TYPE = 'PARQUET');
8
9 -- IMPORTANT:
```

Below the query, the 'Results (5 minutes ago)' panel shows a single row with the status 'Stage area TITANIC_STAGE successfully created.' The 'Query History' panel at the bottom shows the executed query.

The screenshot shows the 'Upload Your Files' dialog in the Snowflake interface. The dialog is titled 'Upload Your Files' and shows a file named 'titanic.parquet' with a size of 39.1KB. Below the file list, there are options to select the database, schema, and stage. The 'Schema' is set to 'DEMO_DB.DEMO_SC' and the 'Stage' is set to 'TITANIC_STAGE'. There is also a section for specifying the path to an existing folder or creating a new one, with an example path '/ Path / To / Folder'. The dialog includes 'Close' and 'Upload' buttons.

Creating table

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

load_to_azure.sql

Monthwise_Hires.sql

Temporary_Table_Queries.sql

Titanic_manual_Load.sql

Titanic_Queries.sql

Titanic_Table_Creation.sql

Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE

TITANIC_STAGE

File Formats 2

INFORMATION_SCHEMA

Home

Create_DB.sql

Monthwise_Hires.sql

Basic_Queries.sql

Verifying_Data_Load.sql

load_to_azure.sql

Tita

ACCOUNTADMIN COMPUTE_WH(X-Small)

DEMO_DB

DEMO_SC

Share

...

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

-- 2. Create the Target Table

CREATE OR REPLACE TABLE titanic_raw (

passenger_id INT,

survived BOOLEAN, -- Loading as Boolean for better Logic later

pclass INT,

full_name STRING,

sex STRING,

age FLOAT,

fare FLOAT

)

-- 3. Load Data using "\$" (Transformation)

Copy data from the Source data into the target object (\$)

Results (just now)

Table Chart

1 row 148ms

Table

status

1 Table TITANIC_RAW successfully created.

Query History

Feedback

Loading data manually into table

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

load_to_azure.sql

Monthwise_Hires.sql

Temporary_Table_Queries.sql

Titanic_manual_Load.sql

Titanic_Queries.sql

Titanic_Table_Creation.sql

Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE

TITANIC_STAGE

File Formats 2

INFORMATION_SCHEMA

Basic_Queries.sql

Verifying_Data_Load.sql

load_to_azure.sql

Titanic_Table_Creation.sql

Titanic_Queries.sql

Titanic_manual_Load

ACCOUNTADMIN COMPUTE_WH(X-Small)

DEMO_DB

DEMO_SC

Share

...

24

25

26

27

28

29

30

31

32

33

34

35

36

37

COPY INTO titanic_raw (passenger_id, survived, pclass, full_name, sex, age, fare)

FROM (

SELECT

\$1:PassengerId::INT,

CASE WHEN \$1:Survived::INT = 1 THEN TRUE ELSE FALSE END, -- Transforming logic during load

\$1:Pclass::INT,

\$1:Name::STRING,

\$1:Sex::STRING,

\$1:Age::FLOAT,

\$1:Fare::FLOAT

FROM @titanic_stage/titanic.parquet

)

FILE_FORMAT = (TYPE = 'PARQUET')

ON_ERROR = 'CONTINUE';

Results (just now)

Table Chart

1 row 1.8s

Table

file status # rows_parsed # rows_loaded # error_limit # errors_seen # first

1 titanic_stage/titanic.pa LOADED 891 891 891 0 null

Query History

Feedback

My Workspace

Basic_Queries.sql Verifying_Data_Load.sql load_to_azure.sql Titanic_Table_Creation.sql Titanic_Queries.sql Titanic_manual_load

Search files Ctrl+P

+ Add new

Basic_Queries.sql
Create_DB.sql
load_to_azure.sql
Monthwise_Hires.sql
Temporary_Table_Queries.sql
Titanic_manual_load.sql
Titanic_Queries.sql
Titanic_Table_Creation.sql
Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE
TITANIC_STAGE
File Formats 2
INFORMATION_SCHEMA

My Workspace > Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB DEMO_SC Share

```
34 FROM @titanic_stage/titanic.parquet
35 )
36 FILE_FORMAT = (TYPE = 'PARQUET')
37 ON_ERROR = 'CONTINUE';
38
39 -- 4. Verify the Load
40 SELECT * FROM titanic_raw LIMIT 10;
41
```

Results (just now)

Table Chart 10 rows 773ms

#	PASSENGER_ID	01 SURVIVED	#	PCLASS	A FULL_NAME	A SEX	#	AGE	#	FARE	
1	10	false	50.0%	1	3	Allen, Mr. William H...	10.0%	fe...	50.0%	7.25	71.2...
2	2	TRUE	50.0%	1	3	Braund, Mr. Owen H...	10.0%	fe...	50.0%	7.25	71.2...
3	3	FALSE	50.0%	1	3	Cummings, Mrs. John Bradley (F...	10.0%	fe...	50.0%	7.25	71.2...

Query History

Creating a view

My Workspace

Basic_Queries.sql Verifying_Data_Load.sql load_to_azure.sql Titanic_Table_Creation.sql Titanic_Queries.sql Titanic_manual_load

Search files Ctrl+P

+ Add new

Basic_Queries.sql
Create_DB.sql
load_to_azure.sql
Monthwise_Hires.sql
Temporary_Table_Queries.sql
Titanic_manual_load.sql
Titanic_Queries.sql
Titanic_Table_Creation.sql
Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE
TITANIC_STAGE
File Formats 2
INFORMATION_SCHEMA

My Workspace > Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB DEMO_SC Share

```
44 CREATE OR REPLACE VIEW v_titanic_summary AS
45 SELECT
46     passenger_id,
47     full_name,
48     sex,
49     age,
50     -- Create a derived column for Age Group
51     CASE
52         WHEN age < 18 THEN 'Child'
53         WHEN age BETWEEN 18 AND 60 THEN 'Adult'
54         ELSE 'Senior'
55     END AS age_category,
56     pclass,
57     fare,
58     survived
59 FROM titanic_raw;
60
```

Results (just now)

Table Chart 1 row 134ms

#	status
1	View V.TITANIC-SUMMARY successfully created.

Query History

My Workspace

Basic_Queries.sql Verifying_Data_Load.sql load_to_azure.sql Titanic_Table_Creation.sql Titanic_Queries.sql Titanic_manual_load

Search files Ctrl+P

+ Add new

Basic_Queries.sql
Create_DB.sql
load_to_azure.sql
Monthwise_Hires.sql
Temporary_Table_Queries.sql
Titanic_manual_load.sql
Titanic_Queries.sql
Titanic_Table_Creation.sql
Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE
TITANIC_STAGE
File Formats 2
INFORMATION_SCHEMA

My Workspace > Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB DEMO_SC Share

```
58 survived
59 FROM titanic_raw;
60
61 -- 6. Test the View
62 SELECT * FROM v_titanic_summary
63 WHERE age_category = 'Child'
64 ORDER BY fare DESC;
```

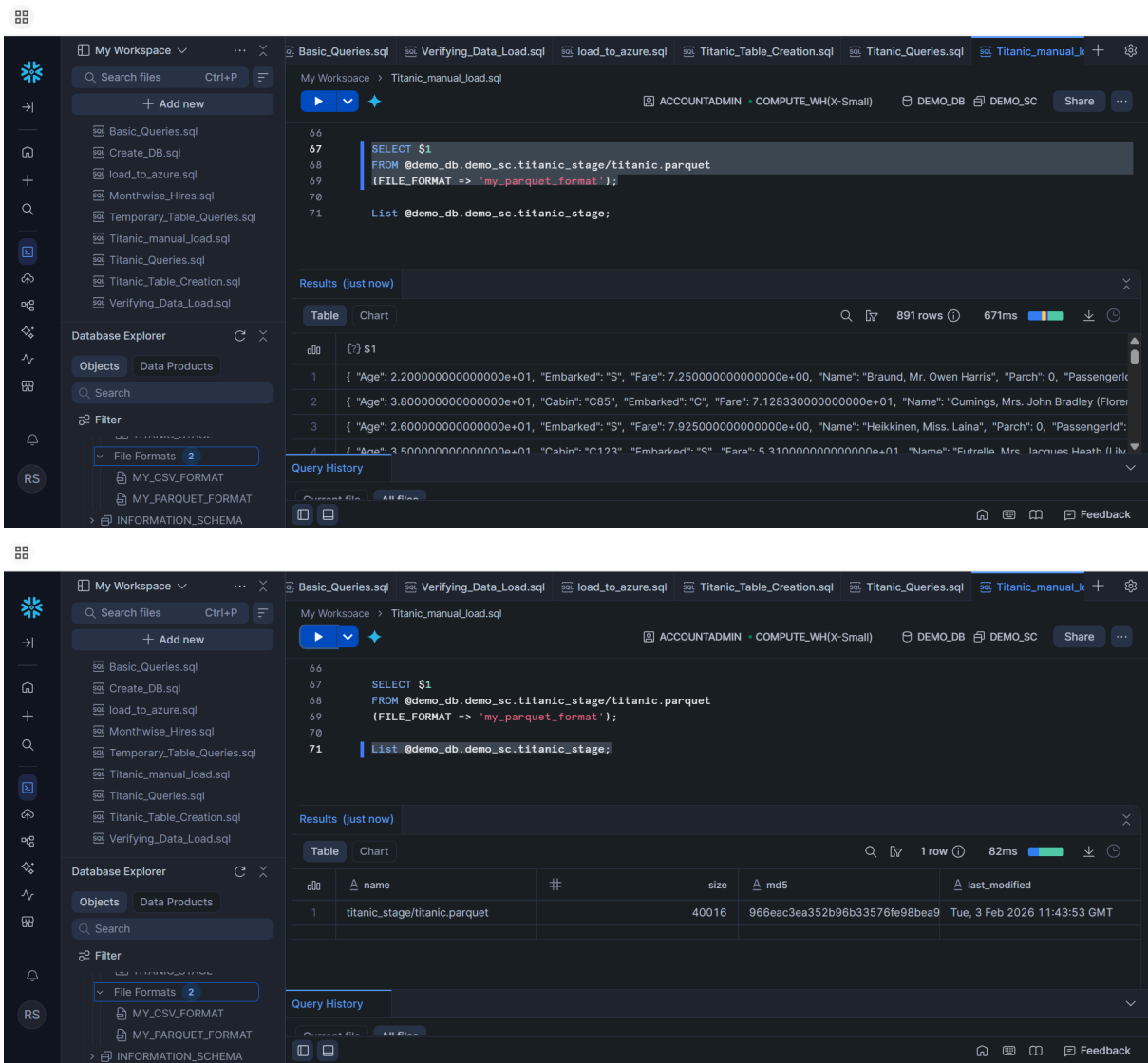
Results (just now)

Table Chart 113 rows 143ms

#	PASSENGER_ID	A FULL_NAME	A SEX	#	AGE	A AGE-CATEGORY	#	PCLASS	#	FARE	01 SURVIVED
1	690	Madill, Miss. George Alexandra	female	15	Child	100.0%	1	3	211.3375	TRUE	
2	298	Allison, Miss. Helen Loraine	female	2	Child	100.0%	1	3	151.55	FALSE	
3	306	Allison, Master. Hudson Trevor	male	0.92	Child	100.0%	1	3	151.55	TRUE	

Query History

Raw stage created



Summary

This project executes two distinct data engineering pipelines: one for a CSV-based Employee dataset and another for a Parquet-based Titanic dataset.

Important Note on Data Ingestion: While the repository includes scripts for cloud integration (Azure) and CLI-based loading, administrative restrictions prevented the use of these methods.

Data was ingested manually via the Snowflake Web UI.

Employee CSVs: Uploaded directly into the target table (EMP).

Titanic Parquet: Uploaded to a Snowflake Stage first, then processed via SQL.

1. Infrastructure & Setup

Create_DB.sql

Purpose: Initializes the foundational database environment.

Operations: Creates the DEMO_DB database and DEMO_SC schema. It defines the EMP table structure (columns for name, email, address, etc.) to prepare for the CSV data load.

load_to_azure.sql

Purpose: Intended to set up a secure connection to external cloud storage.

Operations: Defines a STORAGE INTEGRATION pointing to an Azure Blob container (azure://ronitshettystorage...).

Status: Not executed due to the administrative restrictions mentioned above.

2. Employee Data Pipeline (CSV)

Data Loading (Web UI)

The 5 separate CSV files (employees01–employees05) were manually uploaded via the Web UI directly into the EMP table defined in Create_DB.sql.

Verifying_Data_Load.sql

Purpose: Quality assurance immediately following the manual upload.

Operations: Checks record counts (COUNT(*)), retrieves a sample of rows (LIMIT 10), and verifies string formatting by concatenating names to ensure columns mapped correctly.

Basic_Queries.sql

Purpose: Fundamental exploration of the employee roster.

Operations: Performs standard filtering (finding employees in specific cities), sorting (alphabetical lists), and aggregation (counting employees per city to identify hubs).

Temporary_Table_Queries.sql

Purpose: Advanced, session-based analytics using temporary storage.

Operations: Creates a TEMPORARY TABLE (emp_analysis_temp) that enriches the raw data with calculated columns: email_domain (parsed via SPLIT_PART) and days_employed (calculated via DATEDIFF).

Executes complex logic on this temporary data, such as categorizing staff as 'Veteran' or 'Newcomer', analyzing seasonal hiring (Q4), and filtering via wildcards (LIKE '%Apt%').

Monthwise_Hires.sql

Purpose: Time-series reporting.

Operations: Aggregates hiring data specifically for the year 2017, grouping by MONTHNAME to visualize recruitment spikes.

3. Titanic Data Pipeline (Parquet)

Titanic_Table_Creation.sql

Purpose: Environment setup specific to binary data.

Operations: Defines a custom file format `my_parquet_format` to handle Parquet compression and creates the target schema for the titanic table.

Titanic_manual_load.sql

Purpose: The core ELT (Extract, Load, Transform) script.

Operations: Creates a named internal stage `titanic_stage`.

Context: The `titanic.parquet` file was uploaded to this stage via the Web UI.

Executes a `COPY INTO` command that transforms the data during the load. It queries the staged file using variant syntax (`$1:Survived`), converts data types (e.g., Integer 1/0 to Boolean True/False), and inserts the cleaned data into `titanic_raw`.

Creates a View `v_titanic_summary` to abstract business logic, such as categorizing passengers into 'Child', 'Adult', or 'Senior' based on age.

Titanic_Queries.sql

Purpose: Deep analytical insights into the passenger manifest.

Operations: Calculates survival rates broken down by Gender and Class, computes average ages of survivors vs. victims, analyzes ticket costs, and determines family sizes by adding sibling/spouse and parent/child counts.

Note on Preliminary Data Stage:

The preliminary data loaded is currently in its raw state (Bronze Layer). It was inspected directly from the stage using the following commands:

```
SELECT $1
FROM @demo_db.demo_sc.titanic_stage/titanic.parquet
(FILE_FORMAT => 'my_parquet_format');
```

```
List @demo_db.demo_sc.titanic_stage;
```

The subsequent Silver (cleaning and standardization) and Gold (aggregation and modeling) stages will be performed later.